JOMHACK
EMPOWERING INNOVATION

```
> bootcamp.learn()
```

JOMHACK
EMPOWERING INNOVATION

JomCode Coding Bootcamp

CHAPTER 11

HTTP, JSON and API fundamentals

`learn / type / run / explain`

Plain meaning and technical explanation

PLAIN MEANING

A client asks a server to perform an operation and receives a response.

TECHNICAL EXPLANATION

HTTP requests contain a method, URL, headers and sometimes a body. Responses contain a status, headers and body. APIs define the contract.

REAL-WORLD USE

Understand how Swagger, React and FastAPI communicate.

QUICK TIP

A successful network connection can still return an HTTP error. Check the status.

INSTALL / RUN

No packages. Windows: py workbook.py. macOS: python3 workbook.py. Optional live checks need the Chapter 14 server.

Worked example 1: Parts of a URL

workbook.py

```
01  from urllib.parse import urlparse, parse_qs
02  url = urlparse("http://localhost:8000/posts?limit=2")
03  print(url.scheme, url.hostname, url.port)
04  print(url.path)
05  print(parse_qs(url.query))
```

PRO TIP localhost refers to the computer running the client.

Run and explain example 1

EXPECTED RESULT

http localhost 8000
/posts
{'limit': ['2']}

WHY IT BEHAVES THIS WAY

The path names a resource. Query parameters modify the request and arrive as text.

Worked example 2: Methods and intent

workbook.py

```
operations = {"GET": "read", "POST": "create",
              "PUT": "replace/update", "DELETE": "remove"}
for method, intent in operations.items():
    print(method, intent)
```

PRO TIP GET should not change server data.

Run and explain example 2

EXPECTED RESULT

GET read
POST create
PUT replace/update
DELETE remove

WHY IT BEHAVES THIS WAY

The same /posts path can support several operations distinguished by method.

Worked example 3: Status and body

workbook.py

```
import json
status = 201
body = '{"id": "p1", "title": "Hello"}'
if 200 <= status < 300:
    post = json.loads(body)
    print(post["title"])
print(status == 204)
```

PRO TIP Do not parse an empty 204 response as JSON.

Run and explain example 3

EXPECTED RESULT

Hello
False

WHY IT BEHAVES THIS WAY

201 means creation succeeded. 204 means success with no response body.

Worked example 4: Headers and origins

workbook.py

```
headers = {"Content-Type": "application/json"}
origin = "http://localhost:5173"
api_origin = "http://localhost:8000"
print(headers["Content-Type"])
print(origin == api_origin)
```

PRO TIP Authorization carries a bearer token. Do not print real tokens.

Run and explain example 4

EXPECTED RESULT

application/json
False

WHY IT BEHAVES THIS WAY

Different ports create different origins. Browsers enforce CORS; terminal clients do not.

Exercise 1: Response classifier

YOUR TASK

Classify statuses as success, client error or server error.

CHECK THESE CASES

200 and 204 succeed. 401 is a client error. 500 is a server error.

HINT AFTER A TRY

Compare integer ranges.

Exercise 2: API contract

YOUR TASK

Model the expected response to GET /posts and the empty response to DELETE.

CHECK THESE CASES

GET returns a list of post objects. DELETE 204 has no JSON body.

HINT AFTER A TRY

Represent the examples using Python values.

Checkpoint before moving on

01 Run and explain all four worked examples. Match their actual results to the expected behavior.

02 Attempt both exercises before reading the solutions. Verify every acceptance case.

03 Explain how this chapter supports Postlight: Understand how Swagger, React and FastAPI communicate.